ChatGPT

I've discovered a problem with TVMaze. When you search for something on their website that returns more than 10 results, they are divided into pages, with 10 results per page. But via the API you only get the first 10 results with no apparent way to get the rest of the pages. Can you please investigate, confirm this limitation, and then evaluate alternative services if confirmed?

I investigated this pretty thoroughly, and I think you've found a genuine limitation rather than a bug.

TVMaze Search API

The documented search endpoint is simply:

```
GET /search/shows?q=<query>
```

It accepts only a `q` parameter. There are **no documented `page`, `offset`, `limit`, `skip`, or `cursor` parameters**. TVmaze

The documentation says:

> "Search through all the shows in our database by the show's name... Results are returned in order of relevancy."

It does **not** mention pagination or retrieving additional results. TVmaze

What this means

If a search for "Star" returns 100+ shows on the website:

```
Website:
Page 1: 10 shows
Page 2: 10 shows
Page 3: 10 shows
...

API:
/search/shows?q=star
↓
10 shows only
```

There is no documented way to ask for page 2.

Why does the website have pages?

Almost certainly because the website is **not using the public API endpoint**.

The TVMaze website is backed by its own internal application and database. The public API is intentionally much smaller and simpler. Many websites expose only a subset of their internal capabilities via public

APIs.

So the existence of pagination on the website unfortunately does **not** imply that pagination exists in the API.

How bad is this for NextSeason?

I actually think it depends on your goals.

Typical user behavior

Most users search for:

```
Friends
Lost
Wednesday
Doctor Who
Severance
```

The desired result is almost always in the top few matches.

In fact, TVMaze sorts by relevance, which is generally very good.

For an app whose purpose is:

> "Find a show and add it to my watchlist"

this limitation might never be noticed by 99% of users.

Where it breaks down

Suppose someone searches:

```
Christmas
```

or

```
Love
```

or

```
The
```

There may be hundreds of matches.

If the show they want is ranked #27, your app will never find it.

That isn't ideal.

Possible workarounds

Option 1: Keep TVMaze (my favorite)

Continue using TVMaze.

If search returns 10 results:

```
Searching for "Love"

Found 10 best matches.

Can't find your show?
Try adding another word.
```

For example:

```
Love
↓
Love Island
Love Boat
Love Actually
```

This is actually how many apps work.

Pros

- Free
- Excellent episode data
- Excellent next-episode data
- No backend required
- Already integrated

Cons

- Search is occasionally frustrating.

Option 2: Download the show index

TVMaze provides a **Show Index** endpoint that lets you iterate through every show in the database. They explicitly recommend this approach for some applications. TVmaze +1

You could:

```
nightly sync
    ↓
local SQLite database
    ↓
your own search
```

Advantages:

- Instant search
- Unlimited results
- Better typo matching
- Full control

Disadvantages:

- Tens of thousands of shows
- More code
- Need update logic
- Larger app/database or backend

For an MVP I would not do this.

Alternative APIs

Here are the ones I'd seriously consider.

1. TMDb (The Movie Database) ⭐⭐⭐⭐⭐

Probably the strongest overall alternative.

Pros:

- Excellent search
- Proper pagination
- Huge database
- Movies + TV
- Excellent images
- Active development
- Large community

Cons:

- Requires API key
- Attribution requirements
- Slightly more complex API

It also has a very rich ecosystem and many existing Swift libraries.

---

2. Trakt ⭐⭐⭐⭐☆

Pros:

- Designed for tracking watched shows
- Good search
- User accounts
- Sync across devices

Cons:

- More focused on user tracking than metadata
- API quotas
- Requires authentication for many features

Probably overkill for NextSeason.

---

3. TMDB + TVMaze hybrid ⭐⭐⭐⭐⭐

This is actually the architecture I would seriously consider.

```
User types search
        ↓
TMDb search API
        ↓
Excellent paginated results
        ↓
User selects show
        ↓
Store TVMaze ID (or map it)
        ↓
Use TVMaze for episode/season updates
```

Best of both worlds.

My recommendation

For **NextSeason specifically**, I would **not switch APIs yet**.

Reasons:

MVP

Your app is about:

> "Tell me when my favorite show gets a new season."

It is **not** a TV discovery app.

The search screen is used once per show.

Simplicity

TVMaze gives you:

- next episodes
- seasons
- status
- schedule
- cast
- no authentication
- free access
- straightforward JSON

Those are all major advantages.

UX mitigation

If users only get 10 results, you can present:

> **Can't find your show?**

Try searching for a more specific title, such as "Star Trek Voyager" instead of "Star".

That is a perfectly reasonable MVP tradeoff.

My longer-term recommendation

I would keep TVMaze for the MVP and the resume version.

If NextSeason gains real users or you decide to commercialize it, **TMDb is the first alternative I'd evaluate**. It offers much stronger search capabilities while remaining widely used, well documented, and friendly to independent developers.

Sources

Uploaded a file

Cursor has modified the Search section of Release Readiness, indicating that most of the improvements we documented are not necessary. Please read it again so you have the latest version, and then add your suggestion of what to do if the user's show isn't found.

Also please modify Roadmap.md, which has not changed since you wrote it, to update the search improvements there and add your suggestion of using both TVMaze and TVdB in the future.

I like Cursor's revision quite a bit. It moves the document from "we should fix search" to "let's understand what TVMaze already does and only solve real problems."

Release Readiness.md

I would make only one addition.

Add a new subsection after "Real app-side opportunities"

Markdown

```
### Search Fallback UX

TVMaze's public search API returns a maximum of 10 results with no pagination,
even though the website exposes additional pages.

If a user cannot find the desired show, provide a helpful empty/failure state
instead of implying that no matching show exists.

Potential copy:

> Can't find your show?
>
> Try searching for a more specific title (for example, "Star Trek Voyager"
> instead of "Star"), or include a year or subtitle.

This is a low-cost usability improvement that addresses the API limitation
without adding significant complexity.
```

I would **not** make this a High Priority task. It is inexpensive and worthwhile, but it's really polish rather than functionality.

Roadmap.md

I would update the Search section substantially.

Search Improvements

Replace any discussion of punctuation handling, article stripping, alternate names, and ranking improvements with something like:

Markdown

```
### Search

TVMaze already provides fuzzy matching, alternate-name (AKA) support,
partial-title matching, punctuation tolerance, and relevance-based ranking.

Future work should therefore focus on features that add value beyond the
underlying API rather than duplicating its behavior.

Potential improvements:

- Support common abbreviations and acronyms (GoT, SVU, TNG, etc.).
- Experiment with app-specific result ordering (for example, prioritize
  continuing shows or shows with known upcoming seasons).
- Collect beta analytics before investing additional engineering effort.
```

New roadmap item

I'd also add a new item under a Future Enhancements / Infrastructure section:

Markdown

```
### Evaluate Multi-Provider Search

TVMaze's public search API is limited to 10 results with no pagination,
while the TVMaze website supports additional internal pages.

If search quality becomes a meaningful user pain point, investigate a
multi-provider architecture:

- Use TMDb (or TVDB) for user-facing search.
- Continue using TVMaze for season, episode, and next-airing metadata.
- Map provider IDs when a show is selected.

Benefits:

- Unlimited paginated search results.
- Better discovery of obscure shows.
- Preserve the existing notification and season-tracking implementation.

This should only be pursued if beta feedback demonstrates that the current
TVMaze search limitations materially impact users.
```

One small recommendation

I would actually recommend **TMDb rather than TVDB** as the future search provider.

Reasons:

- Better search experience.
- More mature and widely used API.
- Proper pagination.
- Larger community and better tooling.
- Excellent artwork and metadata if you ever expand the app.

So my roadmap wording would intentionally say:

> **"Use TMDb (or another search-focused provider) for search while continuing to use TVMaze for tracking data."**

That keeps the roadmap flexible while preserving the excellent season/episode information that makes TVMaze attractive for NextSeason.

Sounds good, go ahead.

Done!

I made one small change from my previous suggestion: I changed the future provider recommendation from **TVDB** to **TMDb (or another search-focused provider)** because I think that's the stronger long-term direction while still leaving the roadmap flexible.

I've prepared two Markdown snippets that you can drop into your existing documents:

- **ReleaseReadiness_Search_Addition.md** – adds the user-facing fallback when a show isn't found.
- **Roadmap_Search_Update.md** – updates the search roadmap to reflect TVMaze's existing capabilities and adds the future multi-provider search strategy.

One additional thought that occurred to me while writing this: **don't optimize until you have data**. If you add a single analytics event like:

```
search_performed
query_length
result_count
show_selected (yes/no)
```

you'll very quickly learn whether this limitation is affecting real users. If 99.5% of searches end with a show being selected, there's no reason to build a more complicated search architecture. If you see a significant number of searches with 10 results and no selection, you'll have concrete evidence that it's time to revisit the roadmap.

I think that's exactly the kind of pragmatic, product-driven decision making that will make NextSeason a stronger portfolio project. ‹/›

Please add that recommendation to your updated Roadmap document.

Absolutely. I actually like that addition better than the provider discussion because it reinforces a product philosophy:

> **Measure first, optimize second.**

I've incorporated it as a new **Recommended Analytics** section ahead of the multi-provider proposal and updated the final recommendation to say that the architecture change should be driven by **beta feedback and analytics**, not just intuition.

You can download the updated document here:

Roadmap_Search_Update.md

I think this version tells a stronger story to both future-you and potential employers: it demonstrates that you're making engineering decisions based on user behavior rather than prematurely adding complexity.